Reinforcement Learning Portfolio Management

Project Progress Report

RL Portfolio Management Project

October 15, 2026

1 Introduction

[THOMAS]

This project investigates whether a reinforcement learning agent can improve portfolio allocation across U.S. equity sectors. The agent observes historical market signals and its current holdings, then adjusts sector weights every five trading days. Portfolio returns, trading costs, borrowing costs, and weight drift are calculated daily. The objective is to maximize portfolio growth after the modeled implementation costs.

The current implementation is a research pipeline consisting of market-data collection, feature engineering, a custom portfolio environment, PPO training, saved model checkpoints, and development-validation comparisons against simple benchmarks. Three main model stages have been implemented: a long-only allocator, an unscaled long-short allocator, and a long-short allocator with scaled changes to portfolio weights.

The downloaded market history ends on September 30, 2026. All performance figures below are historical development-validation results used during model development as the ETF experiments reserve the final two years for a later evaluation.

2 Technology Used

The project uses Python and Jupyter notebooks, with PyTorch supporting the neural networks and Stable-Baselines3 providing the Proximal Policy Optimization (PPO) implementation. Gymnasium supplies the interface for the custom allocation environment. NumPy and pandas handle numerical operations, date alignment, feature construction, and saved results. yfinance retrieves Yahoo Finance data. Matplotlib supports the analysis plots.

The workflow is organized into six numbered notebooks: data download, feature engineering, agent training, benchmark analysis, action-scale search, and the preliminary stock-fund experiment. The active settings and action/reward functions are defined in `notebooks/experiment_config.py`. Saved experiment folders contain checkpoints, configuration snapshots, chronological split summaries, daily validation records, and aggregate metrics.

Component	Pinned version	Role
Python	3.11 (tested)	Runtime documented in the requirements file
NumPy / pandas	1.26.4 / 2.2.3	Arrays, feature panels, and performance analysis
yfinance	0.2.61	Adjusted prices and preliminary stock screening
Gymnasium	0.29.1	Observation, action, and episode interfaces
Stable-Baselines3	2.3.2	PPO training and deterministic evaluation
PyTorch	2.4.0	Neural-network training
Matplotlib	3.9.2	Return, drawdown, and allocation plots
JupyterLab / ipykernel	4.2.5 / 6.29.5	Interactive experimentation

Table 1: Dependencies recorded in `requirements.txt`.

3 Data

3.1 Market history and investment universe

The main dataset contains daily observations for the nine original sector ETFs, plus SPY and the VIX as context series. SPY and VIX provide information to the policy but are not tradable assets in the current environment.

ETF	Sector
XLB	Materials
XLE	Energy
XLF	Financials
XLI	Industrials
XLK	Information Technology
XLP	Consumer Staples
XLU	Utilities
XLV	Health Care
XLY	Consumer Discretionary

Table 2: The current nine-asset allocation universe.

To run the analysis, you must clone the GitHub repository and run the cells in `01_download_data.ipynb` The download uses split- and dividend-adjusted prices (`auto_adjust=True`), with open, high, low, close, volume, and corporate-action fields retained. The simulator uses adjusted close-to-close returns.

3.2 Features and observations

Let $P_{i,t}$ denote the adjusted closing price of asset i on trading day t, and let $N = 9$ be the number of tradable sector ETFs. All windows below count trading days: 5 days approximate one week, 21 days one month, and 63 days one quarter. These horizons provide different views of recent market behavior.

[ADD MORE?]

Features for each tradable ETF. Each ETF contributes the following eight features:

- **5-day return (`return_5d`).**

$$r_{i,t}^{(5)} = \frac{P_{i,t}}{P_{i,t-5}} - 1.$$

This measures the price change over approximately one trading week.

- **21-day return (`return_21d`).**

$$r_{i,t}^{(21)} = \frac{P_{i,t}}{P_{i,t-21}} - 1.$$

This measures approximately one month of performance.

- **63-day return (`return_63d`).**

$$r_{i,t}^{(63)} = \frac{P_{i,t}}{P_{i,t-63}} - 1.$$

This measures approximately one quarter of performance.

- **21-day annualized volatility (`volatility_21d`).**

First define the daily log return $\ell_{i,t} = \log(P_{i,t}/P_{i,t-1})$. Then

$$\bar{\ell}_{i,t} = \frac{1}{21}\sum_{k=0}^{20} \ell_{i,t-k}, \qquad \sigma_{i,t}^{(21)} = \sqrt{252}\sqrt{\frac{1}{20}\sum_{k=0}^{20}(\ell_{i,t-k} - \bar{\ell}_{i,t})^2}.$$

This measures recent return variability. The one-month window keeps the estimate responsive to recent conditions with annualization putting it on a familiar yearly scale.

- **Drawdown relative to the 63-day high (`rolling_drawdown_63d`).**

$$D_{i,t}^{(63)} = \frac{P_{i,t}}{\max_{0\le k\le 62} P_{i,t-k}} - 1.$$

This describes how far a sector has fallen from its recent peak. The quarterly window gives context beyond the most recent month. This feature is the current distance from a rolling high.

- **Cross-sectional rank of the 21-day return (`cross_sectional_rank_return_21d`).**

$$Q_{i,t} = \frac{\mathrm{rank}_{j=1,\dots,N}(r_{j,t}^{(21)})_i}{N}.$$

This identifies relative sector leadership even when all sectors rise or fall together.

- **21-day return relative to SPY (`market_relative_return_21d`).**

$$M_{i,t}^{(21)} = r_{i,t}^{(21)} - r_{\mathrm{SPY},t}^{(21)}.$$

This measures whether a sector has outperformed or underperformed the broad market over the month. The feature helps separate sector leadership from a market-wide rally.

- **63-day rolling market beta (`rolling_market_beta_63d`).**

$$\beta_{i,t}^{(63)} = \frac{\mathrm{Cov}_{63}(r_i^{(1)}, r_{\mathrm{SPY}}^{(1)})_t}{\mathrm{Var}_{63}(r_{\mathrm{SPY}}^{(1)})_t}, \qquad r_{i,t}^{(1)} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

Covariance and variance use the latest 63 daily simple returns, including day t, with sample estimates. The quarterly window provides more observations for estimating this relationship than a one-month window.

SPY context features. SPY contributes its 5-, 21-, and 63-day returns, 21-day annualized volatility, and 63-day rolling drawdown. These five inputs describe broad-market trend, variability, and distance from recent highs, helping the policy interpret sector signals in their market context.

Market-wide features. The following five features summarize the sector universe and the volatility environment:

- **Sector breadth above the 63-day moving average (`sector_breadth_above_sma_63d`).**

$$\mathrm{SMA}_{i,t}^{(63)} = \frac{1}{63}\sum_{k=0}^{62} P_{i,t-k}, \qquad B_t = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\Big\{P_{i,t} > \mathrm{SMA}_{i,t}^{(63)}\Big\}.$$

  This is the fraction of sectors above their quarterly moving averages. Values near one indicate widespread participation in an upward trend with values near zero indicate broad weakness.

- **Dispersion of 21-day sector returns (`sector_return_dispersion_21d`).**

$$\bar{r}_t^{(21)} = \frac{1}{N}\sum_{i=1}^{N} r_{i,t}^{(21)}, \qquad S_t = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(r_{i,t}^{(21)} - \bar{r}_t^{(21)})^2}.$$

  This is the sample standard deviation across the nine sectors' monthly returns on the same date. It measures return differences across sectors, whereas each ETF's volatility measures variation through time.

- **Average pairwise 63-day sector correlation (`sector_average_correlation_63d`).**

$$C_t = \frac{2}{N(N-1)}\sum_{i<j} \mathrm{Corr}_{63}(r_i^{(1)}, r_j^{(1)})_t.$$

  This describes how strongly sectors move together. Higher correlation suggests less diversification across sectors, while lower correlation indicates more independent movements.

- **VIX level (`vix_level`).**

$$V_t = \mathrm{Close}_{\texttt{-VIX},t}.$$

  This takes the downloaded VIX closing index level directly, without converting it into an asset return.

- **5-day VIX return (`vix_return_5d`).**

$$v_t^{(5)} = \frac{V_t}{V_{t-5}} - 1.$$

  This is the proportional change in the VIX index over one trading week. It is an index change, rather than the return from an investable VIX position.

Portfolio accounting and observation preparation.

- **One-day simple return (`return_1d`).**

$$r_{i,t}^{(1)} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

  This is stored to calculate daily portfolio returns and weight drift, and it underlies beta and correlation estimates.

- **Trailing standardization and clipping.** For each market-feature column x_t, set $W = 1{,}260$ and compute

$$\mu_t = \frac{1}{W}\sum_{k=1}^{W} x_{t-k},$$
$$s_t = \sqrt{\frac{1}{W}\sum_{k=1}^{W}(x_{t-k} - \mu_t)^2},$$
$$z_t = \min\left(10, \max\left(-10, \frac{x_t - \mu_t}{s_t}\right)\right).$$

  The scaling code uses a population standard deviation (`ddof=0`), unlike the sample standard deviations used in the volatility and dispersion features. Scaling puts inputs with different units, such as VIX levels and decimal returns, on comparable numerical scales. The five-year window provides a substantial historical reference that adapts over time. Statistics exclude the current observation and all future observations, and the same trailing procedure is used during training and evaluation. Clipping bounds extreme standardized values to support numerical stability; it does not remove their observations.

- **Current portfolio weights.** The nine current sector weights w_{i,t^-} are appended directly to the scaled market features without trailing standardization. They show the policy its existing allocation, including any negative positions, so residual actions can depend on current holdings and the cost of changing them.

3.3 Chronological partitions

The most recent two years are reserved as a final holdout. Earlier observations are split chronologically, with 80% assigned to training before the scaling warm-up is removed. Consequently, the usable training rows are fewer than 80% of the final, standardized development dataset.

Partition	First date	Last date	Dates
Training	2004-04-08	2019-08-23	3,871
Development validation	2019-08-26	2024-09-30	1,283
Reserved final holdout	2024-10-01	2026-09-30	501

Table 3: Partitions recorded in the saved model split files.

Validation starts with an observation on August 26, 2019, so the saved return series contains 1,282 realized daily returns from August 27, 2019 through September 30, 2024. The tables in this report use that common return period.

4 Training

4.1 Proximal Policy Optimization (PPO)

[ETHAN]

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm that learns a decision rule, called a *policy*, through interaction with an environment. In this project, the environment is the historical portfolio simulator: the agent observes market features and current holdings, proposes allocation changes, and receives a reward based on the resulting portfolio growth after costs. Training seeks to improve expected discounted cumulative reward across successive decisions.

PPO uses an *actor–critic* approach:

- **Actor (policy).** The policy $\pi_\theta(a \mid s)$ defines a distribution over actions given observation s, with trainable parameters θ. Sampling actions during training provides exploration. Here, the sampled actions are converted into changes in the nine sector weights.
- **Critic (value function).** The value function $V_\phi(s)$ estimates the expected discounted future reward from observation s. It supplies a baseline for assessing actions, and is trained against estimated return targets.
- **Advantage estimate.** The estimate $\widehat{A}_k$ measures how favorable an action's outcome was relative to the critic's baseline, using subsequent rewards and value estimates. Positive advantages encourage the actor to make those actions more likely; negative advantages encourage the opposite. Here, k indexes policy decisions, each normally spanning five trading days.

PPO collects a batch of experience with the current policy, then performs multiple minibatch optimization passes before collecting a new batch. Its clipped policy objective discourages excessive changes to action likelihoods during those updates. Define the likelihood ratio [VERIFY THIS]

$$q_k(\theta) = \frac{\pi_\theta(a_k \mid s_k)}{\pi_{\theta_{\text{old}}}(a_k \mid s_k)}.$$

[VERIFY THIS] For continuous actions, the numerator and denominator are probability densities. The actor maximizes

$$L^{\text{clip}}(\theta) = \widehat{\mathbb{E}}_k \left[\min\Big(q_k(\theta)\widehat{A}_k,\ \text{clip}(q_k(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_k \Big) \right],$$

where ϵ controls clipping and $\widehat{\mathbb{E}}_k$ denotes the batch average. The minimum removes the incentive to push an already favorable likelihood change beyond the clipping threshold. This encourages conservative updates but does not impose a hard bound on total policy change.[1]

[VERIFY THIS] PPO is an on-policy method: new experience is generated with the latest policy after each update cycle. The Stable-Baselines3 implementation also normalizes advantages during training.[2] We use it as a practical starting point because it supports continuous actions and integrates with our Gymnasium simulator.

4.2 Learning setup

All models use PPO with a feedforward multilayer-perceptron policy (`MlpPolicy`). The policy outputs nine continuous actions in $[-1, 1]$. The model-specific action function converts these actions into changes in portfolio weights. Episodes begin with an equal-weight portfolio and sample random

[1] Schulman et al. (2017), *Proximal Policy Optimization Algorithms*, https://arxiv.org/abs/1707.06347.

[2] Stable-Baselines3 2.3.2, PPO documentation: https://stable-baselines3.readthedocs.io/en/v2.3.2/modules/ppo.html.

starting points within the training partition. Each episode spans up to five trading years (1,260 daily intervals), while the policy acts once every five trading days.

[HYPERPARAMETERS THAT COULD BE CHANGED]

Setting	Current value
Requested training budget	300,000 environment steps per run
Learning rate	3×10^{-4}
PPO rollout length	2,048 environment steps
Minibatch size	64
Episode length	Five trading years, with random training starts
Decision interval	Five trading days; daily valuation and accounting
Initial main-model seed	42
Repeated-seed experiments	42, 7, 21, 123, and 2026
Evaluation policy	Deterministic actions over development validation

Table 4: Shared training settings. An environment step normally spans five trading days; it is not a single daily return. PPO can round the requested training budget up to finish a rollout.

4.3 Portfolio accounting and reward

Let w_t be the weights established for a daily return interval, w_{t^-} the weights before trading, and r_{t+1} the vector of next-day asset returns. Turnover is the full sum of absolute weight changes, counting both purchases and sales:

$$\tau_t = \sum_{i=1}^{N} |w_{i,t} - w_{i,t^-}|$$

[ASSUMING TRANSACTION COSTS AND BORROWING, COULD BE CHANGED]

The environment charges a transaction-cost rate $c = 0.0005$ (5 basis points per dollar traded), plus a daily borrowing charge based on an annual rate $b = 0.01$ applied to short exposure. Net return and the daily reward are

$$r_{t+1}^{\text{net}} = w_t^\top r_{t+1} - c\tau_t - \frac{b}{252} \sum_i \max(-w_{i,t}, 0),$$

$$R_{t+1} = 100 \log\left(1 + r_{t+1}^{\text{net}}\right).$$

The PPO step reward sums these daily rewards across the decision interval. The objective therefore rewards compounded growth after costs, with no separate drawdown or volatility penalty. Between decisions, holdings drift with asset returns rather than being reset to constant weights every day.

Performance reporting includes compounded net return, annualized net return, annualized volatility, Sharpe ratio, maximum drawdown, turnover, exposure, and implementation costs. Annualization uses 252 trading days. The Sharpe ratio uses a zero risk-free rate. Reported annualized implementation cost is the mean daily transaction-plus-borrow charge multiplied by 252.

5 Model 1: Long Only

[HANK]

The first model learns changes to a fully invested long-only portfolio. For action vector a_t, the proposed weights are

$$\widetilde{w}_t = w_{t^-} + a_t - \overline{a}_t \mathbf{1}.$$

Subtracting the mean action makes the proposed change sum to zero. The model can concentrate in selected sectors, but it cannot short, borrow to increase exposure, or allocate to a cash asset.

The saved seed-42 run in `models/long_only/` achieved a 23.51% annualized net return, a 0.975 Sharpe ratio, and a maximum drawdown of 35.07%. Its annualized turnover was 22.54 times portfolio value, with an annualized implementation cost of 1.13%.

6 Model 2: Long Short

The second model retains the same centered, unscaled residual action but permits negative weights which allows the agent to take short position. Net exposure remains approximately 100%. If a proposed portfolio exceeds 160% gross exposure, its positive positions are rescaled to 130% long exposure and its negative positions to 30% short exposure. When the proposal is already within the gross limit, it is retained. Thus the model can use less than the full short allowance.

The seed-42 run in `models/long_short/` produced a 19.37% annualized net return and a Sharpe ratio of 0.932, with a 33.10% maximum drawdown. Annualized turnover increased to 45.74 times portfolio value, and implementation cost rose to 2.59% per year. Shorting expanded the available portfolios, but the large residual actions made trading expensive, and this run did not improve on the long-only run's return or Sharpe ratio.

The exposure constraint is applied when actions establish new target weights. Daily drift can subsequently push gross exposure above 160%; the original long-short run records a maximum of approximately 164.71%. The current simulator therefore does not enforce a continuous daily gross-exposure ceiling.

7 Model 3: Long Short with Scaled Actions

7.1 Smaller changes to current holdings

The third model introduces an action scale α:

$$\widetilde{w}_t = w_{t^-} + \alpha \left(a_t - \overline{a}_t \mathbf{1} \right).$$

It uses the same long-short exposure mapping as Model 2. Scaling changes how far the policy moves from its current holdings at each decision; it does not impose a fixed final position size or a hard turnover cap.

The scaled run used $\alpha = 0.10$. For seed 42, annualized turnover fell from 45.74 to 16.69, a reduction of approximately 63.5% relative to the unscaled long-short run. Annualized implementation cost fell from 2.59% to 1.13%. Annualized net return increased to 21.97%, and Sharpe increased slightly to 0.936. However, maximum drawdown worsened from 33.10% to 34.74% in magnitude.

Metric	Long only	Long short	Scaled 0.10
Cumulative net return	192.74%	146.12%	174.67%
Annualized net return	23.51%	19.37%	21.97%
Annualized volatility	24.83%	21.47%	24.41%
Sharpe ratio	0.975	0.932	0.936
Maximum drawdown	−35.07%	−33.10%	−34.74%
Annualized turnover	22.54	45.74	16.69
Annualized impl. cost	1.13%	2.59%	1.13%

Table 5: Saved seed-42 development-validation results. The scaled model uses $\alpha = 0.10$. Turnover is expressed as multiples of portfolio value per year.